



Shailesh Yadav-da

Data Analyst

SQL — Complete Course to Transform
Your Data Skills

Welcome To My Post

Scroll More >

CREATE DATABASE

```
CREATE DATABASE company_db;
```

Database use

```
USE company_db;
```

✓ Now whatever table is created will be created in company_db.

Categories of Datatypes

1. **String**

2. **Numeric**

3. **Date & Time**

String Data Type

1. **CHAR** (size) 0 to 255
2. **VARCHAR** (size) 0 to 65535
3. **BINARY** (size)
4. **VARBINARY** (size)
5. **TINYTEXT** 255 Characters
6. **TEXT** (size) 65, 535 bytes
7. **MEDIUM TEXT** 16, 777, 215 Characters
8. **LONGTEXT** 4, 294, 967, 295 Characters
9. **TINYBLOB** 255 bytes
10. **BLOB** (size) 65, 535 bytes
11. **MEDIUMBLOB** 16, 777, 215 bytes
12. **LOB** 4, 294, 967, 295 bytes
13. **ENUM** (val1, val2, val3, ...) list up to 65535 values
14. **SET** (Val1, val2, val3, ...) list up to 64 values

Number Data Type

1. **BIT** (size) 1 to 64
2. **TINYINT** (size) -128 to 127
3. **INT** (size) -2147483648 to 2147483647
4. **INTEGER** (size)
5. **SMALLINT** (size) -32768 to 32767
6. **MEDIUMINT** (size) -8388608 to 8388607
7. **BIGINT** (size) -9223372036854775808 to 9223372036854775807
8. **BOOL**
9. **BOOLEAN** 0/1
10. **FLOAT** (p)
11. **DOUBLE** (size, d) 255.268
12. **DECIMAL** (size, d) size = 60, d = 30
13. **DEC** (size, d)

Date Data Type

1. **DATE** '1000-01-01' TO '999-12-31'
2. **DATETIME** (fsp) YYYY-MM-DD hh: mm: ss
3. **TIMESTAMP** (fsp)
4. **TIME** (fsp) hh:mm:ss
5. **YEAR** four-digit format : 1901

CREATE TABLE



Query

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(30),  
    salary INT,  
    joining_date DATE  
);
```

INSERT DATA (ADD RECORDS)



Query

```
INSERT INTO employees VALUES  
(1, 'Rahul', 'Sales', 35000, '2023-01-10'),  
(2, 'Anita', 'HR', 30000, '2022-07-15'),  
(3, 'Vikas', 'IT', 50000, '2021-03-20');
```

SELECT DATA (VIEW RECORDS)

```
SELECT * FROM employees;
```

WHERE (FILTER DATA)

Concept

Use **WHERE** to apply conditions.

Query

```
SELECT * FROM employees  
WHERE department = 'Sales';
```


CONSTRAINTS

- NOT NULL
- UNIQUE
- DEFAULT
- CHECK
- FOREIGN KEY
- PRIMARY KEY

EXAMPLE

```
CREATE TABLE employees (  
  emp_id SERIAL PRIMARY KEY,      -- Har ID unique hogi aur khali nahi hogi  
  full_name TEXT NOT NULL,        -- Naam likhna compulsory hai  
  email TEXT UNIQUE,              -- Do employees ki email same nahi ho sakti  
  age INT CHECK (age >= 18),      -- 18 se kam age ka entry nahi hoga  
  status TEXT DEFAULT 'Active',   -- Agar kuch na likha toh 'Active' mana jayega  
  dept_id INT REFERENCES departments(id) -- Foreign Key jo dusri table se linked  
  hai  
);
```

SQL LOGICAL OPERATORS

AND • OR • NOT

◆ 1. AND OPERATOR

Use

The AND condition is **TRUE** only when both conditions are TRUE.

◆ Example

```
SELECT *  
FROM employees_full  
WHERE department = 'IT'  
AND experience_years > 5;
```

IT Department + More than 5 years of experience

◆ 2. OR OPERATOR

📌 Use

And it is true when at least **one** condition is true.

◆ Example

Sales or **IT** department employees

```
SELECT first_name, department  
FROM employees_full  
WHERE department = 'Sales'  
      OR department = 'IT';
```

◆ 3. NOT OPERATOR

Use

The **NOT** condition reverses the condition.

◆ Example

```
SELECT first_name, department  
FROM employees_full  
WHERE NOT department = 'Sales';
```

◆ 4. AND + OR COMBINATION
(VERY IMPORTANT)

Sales or IT + salary > 100000

```
SELECT *  
FROM employees_full  
WHERE (department = 'Sales' OR  
department = 'IT')  
AND salary > 100000;
```

SQL IN OPERATOR

◆ What is the IN operator?

The IN operator is used when you need to match a column's value against multiple values — without having to write the same condition repeatedly.

◆ Example : Salary List Match

```
SELECT first_name, salary
FROM employees_full
WHERE salary IN (90000, 100000, 120000);
```

```
SELECT *
FROM employees_full
WHERE department IN ('Sales', 'IT')
AND salary > 100000;
```

SQL LIKE OPERATOR

The LIKE operator is used when we need to search for a pattern in a text (string).

It uses wildcards:

- % → zero ya more characters
 - _ → exactly one character
- ◆ **Example 1:** Name "A" se start hota ho

```
SELECT first_name  
FROM employees_full  
WHERE first_name LIKE 'A%';
```

- ◆ **Example 2:** The name ends with "a".

```
SELECT first_name  
FROM employees_full  
WHERE first_name LIKE '%a';
```

- ◆ **Example 3:** Name ke beech me "sh" ho

```
SELECT first_name  
FROM employees_full  
WHERE first_name LIKE '%sh%';
```

- ◆ **Example 4:** Second Letter "a" ho


```
SELECT first_name  
FROM employees_full  
WHERE first_name LIKE '_a%';
```

◆ Example 5: Email Gmail ka ho

```
SELECT first_name, email  
FROM employees_full  
WHERE email LIKE '%@gmail.com';
```

◆ Example 6: LIKE with AND

```
SELECT first_name, department  
FROM employees_full  
WHERE department = 'Sales'  
      AND first_name LIKE 'A%';
```

SQL BETWEEN / NOT BETWEEN

- ◆ What is the **BETWEEN** operator?

The BETWEEN operator is used when we need to check if a value falls within a specific range (from start to end).

- ◆ Example 1: Salary Range

Salary between Rs 80,000 to Rs 1,20,000

```
SELECT first_name, salary  
FROM employees_full  
WHERE salary BETWEEN 80000 AND 120000;
```

◆ Example 2: Experience Range

```
SELECT first_name, experience_years  
FROM employees_full  
WHERE experience_years BETWEEN 3 AND 6;
```

◆ Example 3: Date Range (Very Important)

```
SELECT first_name, hire_date  
FROM employees_full  
WHERE hire_date BETWEEN '2020-01-01'  
AND '2022-12-31';
```

◆ Example 4: BETWEEN with AND (Department Filter)

```
SELECT first_name, department, salary
FROM employees_full
WHERE department = 'Sales'
AND salary BETWEEN 100000 AND 150000;
```

◆ **NOT BETWEEN Operator**

◆ **Example 5: Salary Range Exclude**

All except 80,000 to 1,20,000

```
SELECT first_name, salary
FROM employees_full
WHERE salary NOT BETWEEN 80000
AND 120000;
```

SQL ORDER BY & DISTINCT

PART-1: ♦ DISTINCT OPERATOR

♦ What is DISTINCT?

DISTINCT is used to remove duplicate values. It is only used with **SELECT** statements.

♦ Example 1: Unique Departments

```
SELECT DISTINCT department  
FROM employees_full;
```

- ◆ **Example 2: DISTINCT with multiple columns**

```
SELECT DISTINCT department, job_title  
FROM employees_full;
```

- ◆ **Example 3: DISTINCT with WHERE**

```
SELECT DISTINCT job_title  
FROM employees_full  
WHERE department = 'Sales';
```

PART-2: ♦ ORDER BY OPERATOR

♦ What is ORDER BY?

ORDER BY is used to sort (arrange) the data.

♦ Example 4: Salary Ascending Order

```
SELECT first_name, salary  
FROM employees_full  
ORDER BY salary ASC;
```

♦ Example 5: ORDER BY with **WHERE**

```
SELECT first_name, salary  
FROM employees_full  
WHERE department = 'IT'  
ORDER BY salary DESC;
```

SQL LIMIT & OFFSET

◆ What is the **LIMIT**?

The LIMIT clause is used to fix the maximum number of output rows.

◆ Example 1: Sirf 5 Records Dikhao

```
SELECT *  
FROM employees_full  
LIMIT 5;
```


◆ Example 2: Top 5 Highest Salary Employees

```
SELECT first_name, salary  
FROM employees_full  
ORDER BY salary DESC  
LIMIT 5;
```

👉 First ORDER BY, then LIMIT

◆ What is OFFSET?

Offset is used to skip the starting rows.

◆ Example 3: First 5 Rows Skip, Next 5
Dikhao

```
SELECT *  
FROM employees_full  
LIMIT 5 OFFSET 5;
```

- ◆ OFFSET Without LIMIT (Not Recommended)

```
SELECT *  
FROM employees_full  
OFFSET 5;
```

SQL AGGREGATE FUNCTIONS

◆ What is an Aggregate Function?

Aggregate functions multiple rows par kaam karke ek single value return karti hain.

◆ Common Aggregate Functions

- **count()**
- **SUM()**
- **AVG()**
- **MIN()**
- **MAX()**

COUNT() Function

◆ Use

How many records are there?

Example 1: Total Employees

```
SELECT COUNT(*) AS total_employees  
FROM employees_full;
```

Example 2: How many people are in the sales department?

```
SELECT COUNT(*)  
FROM employees_full  
WHERE department = 'Sales';
```

SUM() Function

◆ Use

Total value nikalna (salary, sales, amount)

```
SELECT SUM(salary) AS total_salary  
FROM employees_full;
```

Example 3: Sales Department ki total salary

```
SELECT SUM(salary)  
FROM employees_full  
WHERE department = 'Sales';
```

AVG() Function

◆ Use

Average nikalna

Example 4: Average Salary

```
SELECT AVG(salary) AS avg_salary  
FROM employees_full;
```

Example 5: IT Department ki average salary

```
SELECT AVG(salary)  
FROM employees_full  
WHERE department = 'IT';
```

MIN() Function

◆ Use

The smallest value

Example 6: Minimum Salary

```
SELECT MIN(salary) AS min_salary  
FROM employees_full;
```

MAX() Function

◆ Use

The biggest value

Example 7: Maximum Salary

```
SELECT MAX(salary) AS max_salary  
FROM employees_full;
```

◆ Aggregate with GROUP BY + HAVING

Example : Departments jahan avg salary > 100000

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees_full  
GROUP BY department  
HAVING AVG(salary) > 100000;
```


SQL STRING FUNCTIONS

LENGTH() – Find length of a string

```
SELECT first_name, LENGTH(first_name)
FROM employees;
# Rahul | 5
# Neha | 4
```

UPPER() – Convert to uppercase

```
SELECT UPPER(first_name)
FROM employees;
# RAHUL
# NEHA
```

LOWER() – Convert to lowercase

```
SELECT LOWER(email)
FROM employees;
# rahul@gmail.com
# neha@yahoo.com
```

INITCAP() – Capitalize first letter

```
SELECT INITCAP(first_name)
FROM employees;
# Rahul
# Neha
```

CONCAT() – Join strings

```
SELECT CONCAT(first_name, ' ', last_name)
FROM employees;
# Rahul Sharma
# Neha Singh
```

|| (Concatenation Operator)

```
SELECT first_name || ' works in ' || department
FROM employees;
--Rahul works in Sales
--Neha works in IT
```

SUBSTRING() – Extract part of string

```
SELECT SUBSTRING(email FROM 1 FOR 5)
FROM employees;
-- rahul
-- neha@
```

LEFT() – Characters from left side

```
SELECT LEFT(first_name, 3)
FROM employees;
# Rah
# Neh
```

RIGHT() – Characters from right side

```
SELECT RIGHT(last_name, 4)
FROM employees;
# arma
# ingh
```

10 TRIM() – Remove spaces

```
SELECT TRIM('  Rahul  ');  
# Rahul
```

REPLACE() – Replace text

```
SELECT REPLACE(email, 'gmail.com', 'company.com')  
FROM employees;  
# rahul@company.com
```

POSITION() – Find character position

```
SELECT POSITION('@' IN email)  
FROM employees;  
# 6
```

SPLIT_PART() – Split string

```
SELECT SPLIT_PART(email, '@', 1)  
FROM employees;  
# rahul  
# neha
```

REVERSE() – Reverse string

```
SELECT REVERSE(first_name)
FROM employees;
# luhar
# ahen
```

Practical REAL Query (Interview Type)

Find employees whose email domain is gmail.com

```
SELECT first_name, email
FROM employees
WHERE SPLIT_PART(email, '@', 2) = 'gmail.com';
# Rahul | rahul@gmail.com
```

SQL UPDATE QUERY

◆ What is an UPDATE query?

The update statement is used to modify/change the existing data in a table.

 **The WHERE clause is very important.**

(If you don't use the WHERE clause, the entire table will be updated.)

◆ Example 1: Single Column Update

Employee ID = 5 ki salary update karo

```
UPDATE employees_full  
SET salary = 120000  
WHERE employee_id = 5;
```

◆ Example 2: Multiple Columns Update

```
UPDATE employees_full  
SET salary = 90000,  
    department = 'IT'  
WHERE employee_id = 8;
```

◆ Example 3: Condition Based Update

Increase the salaries of the sales department employees by +10,000.

```
UPDATE employees_full  
SET salary = salary + 10000  
WHERE department = 'Sales';
```

◆ Example 4: UPDATE with AND / OR

```
UPDATE employees_full  
SET salary = salary + 5000  
WHERE department = 'HR'  
AND experience_years > 5;
```

◆ Example 5: UPDATE Using BETWEEN

```
UPDATE employees_full  
SET salary = salary + 8000  
WHERE salary BETWEEN 80000 AND 100000;
```


SQL DELETE QUERY

◆ What is a DELETE query?

The DELETE command is used to remove existing rows from a table.

 The WHERE clause is very important.

(If you don't use the WHERE clause, the data from the entire table will be deleted.)

◆ Example 1: Single Record Delete

Delete Employee Id = 10

```
DELETE FROM employees_full  
WHERE employee_id = 10;
```

◆ Example 2: Condition Based Delete

HR department ke employees delete karo

```
DELETE FROM employees_full  
WHERE department = 'HR';
```

◆ Example 5: DELETE Using BETWEEN

```
DELETE FROM employees_full  
WHERE hire_date BETWEEN '2015-01-01' AND  
'2016-12-31';
```

PostgreSQL ALTER COLUMN (Detailed Guide)

What is ALTER COLUMN?

ALTER COLUMN is used with the ALTER TABLE statement to modify an existing column in a table without deleting the table or data.

You can use it to:

- Change a column's data type
- Set or remove DEFAULT values
- Set or remove NOT NULL
- Rename a column
- Change column properties

Change Column Data Type

Syntax

```
ALTER TABLE employees  
ALTER COLUMN salary TYPE BIGINT;
```

Set a DEFAULT Value

```
ALTER TABLE employees  
ALTER COLUMN status SET  
DEFAULT 'Active';
```

Add NOT NULL Constraint

```
ALTER TABLE employees  
ALTER COLUMN email SET NOT NULL;
```

Rename a Column

```
ALTER TABLE employees  
RENAME COLUMN fname TO  
first_name;
```

Change Column Length (VARCHAR)

```
ALTER TABLE employees  
ALTER COLUMN first_name TYPE  
VARCHAR(100);
```

Change Table name

```
ALTER TABLE employee  
RENAME TO EMP
```

When Required (Using USING clause)

```
ALTER TABLE employees  
ALTER COLUMN salary TYPE INTEGER  
USING salary::INTEGER;
```

**Existing NULL values update
karo (IMPORTANT)**

```
UPDATE employees  
SET age = 25  
WHERE age IS NULL;
```

SQL GROUP BY & HAVING

What is GROUP BY?

GROUP BY is used to group rows that have the same values in one or more columns and then apply aggregate functions (COUNT, SUM, AVG, MIN, MAX) on each group.

Simple Example: Department-wise Employee Count

```
SELECT department, COUNT(*) AS  
total_employees  
FROM employees  
GROUP BY department;
```

- ✓ Groups employees by department
- ✓ Counts employees in each department

GROUP BY with SUM

Department-wise Total Salary

```
SELECT department, SUM(salary) AS  
total_salary  
FROM employees  
GROUP BY department;
```

◆ HAVING CLAUSE

What is HAVING?

HAVING is used to filter groups created by GROUP BY.

☞ **WHERE** → filters rows

☞ **HAVING** → filters groups (after aggregation)

Example: Departments with Average Salary > 100000

```
SELECT department, AVG(salary) AS  
avg_salary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 100000;
```

WHERE + GROUP BY + HAVING (Most Important)

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
WHERE status = 'Active'  
GROUP BY department  
HAVING AVG(salary) > 80000;
```

SQL CASE Expression

What is CASE?

CASE is a conditional expression in SQL.

It works like IF–ELSE logic and allows you to return different values based on conditions.

 **CASE is not a statement, it is an expression, so it can be used in:**

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- UPDATE

CASE Syntax (Searched CASE – Most Used)

CASE

WHEN condition1 THEN result1

WHEN condition2 THEN result2

ELSE result

END

Simple Example: Salary Category

SELECT first_name, salary,

CASE

WHEN salary >= 100000 THEN 'High Salary'

WHEN salary >= 60000 THEN 'Medium Salary'

ELSE 'Low Salary'

END AS salary_category

FROM employees;

- ✓ Adds a calculated column
- ✓ Does not change table data

CASE with AND / OR Conditions

```
SELECT first_name, department, salary,  
       CASE  
         WHEN department = 'Sales' AND salary > 80000 THEN 'Top Sales'  
         WHEN department = 'IT' AND salary > 90000 THEN 'Senior IT'  
         ELSE 'Regular Employee'  
       END AS employee_type  
FROM employees;
```

CASE in WHERE Clause (Conditional Filtering)

```
SELECT *  
FROM employees  
WHERE  
  CASE  
    WHEN department = 'HR' THEN salary > 50000  
    ELSE salary > 70000  
  END;
```

☞ HR → salary > 50,000

☞ Others → salary > 70,000

CASE in UPDATE Query

```
UPDATE employees
SET salary = CASE
    WHEN performance_rating = 'A' THEN salary + 20000
    WHEN performance_rating = 'B' THEN salary + 10000
    ELSE salary
END;
```

Common Mistakes (Avoid These)

- ✗ Forgetting END
- ✗ Mixing data types in THEN / ELSE
- ✗ Using CASE instead of WHERE unnecessarily
- ✗ Assuming CASE changes stored data (it doesn't unless used with UPDATE)

Interview One-Liners

- CASE is SQL's IF-ELSE logic
- CASE is an expression, not a statement
- CASE can be used with SELECT, WHERE, ORDER BY, UPDATE

SQL SET OPERATORS

What are Set Operators?

Set operators are used to combine the results of two or more SELECT queries into a single result set.

👉 Think of them like mathematical set operations.

Rules for Using Set Operators (Very Important)

Before using any set operator:

1. Same number of columns in each SELECT
2. Compatible data types in corresponding columns
3. Same column order
4. Column names come from the first SELECT

Types of Set Operators

Operator	Purpose
UNION	Combine + remove duplicates
UNION ALL	Combine + keep duplicates
INTERSECT	Common rows
EXCEPT	Rows in first query, not in second

◆ 1. UNION

What is UNION?

UNION combines results of two SELECT queries and removes duplicate rows.

Example: Employees from Sales and IT

```
SELECT employee_id, first_name  
FROM employees  
WHERE department = 'Sales'
```

UNION

```
SELECT employee_id, first_name  
FROM employees  
WHERE department = 'IT';
```

✓ **Duplicates automatically removed**

◆ 2. UNION ALL

What is UNION ALL?

UNION ALL combines results but keeps duplicate rows.

Example: All Employees from Sales and IT
(Including Duplicates)

```
SELECT employee_id, first_name  
FROM employees  
WHERE department = 'Sales'
```

UNION ALL

```
SELECT employee_id, first_name  
FROM employees  
WHERE department = 'IT';
```

- ✓ **Faster than UNION**
- ✓ **No duplicate check**

◆ 3. INTERSECT

What is INTERSECT?

INTERSECT returns only the rows that appear in both queries.

Example: Employees in BOTH Sales and Management

```
SELECT employee_id  
FROM sales_team
```

```
INTERSECT
```

```
SELECT employee_id  
FROM management_team;
```

✓ **Only common employee IDs**

◆ 4. EXCEPT

What is EXCEPT?

EXCEPT returns rows that are in the first SELECT but NOT in the second.

Example: Employees in Sales but NOT in IT

```
SELECT employee_id  
FROM employees  
WHERE department = 'Sales'
```

EXCEPT

```
SELECT employee_id  
FROM employees  
WHERE department = 'IT';
```

Interview One-Liners

- UNION removes duplicates
- UNION ALL is faster
- INTERSECT returns common rows
- EXCEPT returns difference



DATE & TIME IN POSTGRESQL

Current Date & Time

```
SELECT CURRENT_DATE;
```

```
# 2026-01-10
```

```
SELECT CURRENT_TIME;
```

```
# 20:15:30.123456
```

```
SELECT NOW();
```

```
# 2026-01-10 20:15:30.123456
```

Date vs Timestamp

```
SELECT hire_date, created_at
```

```
FROM employees;
```

```
# 2022-06-15 | 2022-06-15 09:30:00
```

- DATE → only date
- TIMESTAMP → date + time

Extract Parts from Date

```
SELECT  
    EXTRACT(YEAR FROM hire_date),  
    EXTRACT(MONTH FROM hire_date),  
    EXTRACT(DAY FROM hire_date)  
FROM employees;  
# 2022 | 6 | 15
```

AGE() – Difference Between Dates

```
SELECT AGE(CURRENT_DATE, hire_date)  
FROM employees;  
# 3 years 6 mons 25 days
```

Date Arithmetic (Add / Subtract)

```
SELECT hire_date + INTERVAL '1 year'  
FROM employees;  
# 2023-06-15
```

```
SELECT hire_date - INTERVAL '10 days'  
# 2022-06-05
```

TO_CHAR() – Format Date (MOST IMPORTANT)

```
SELECT TO_CHAR(hire_date, 'DD-Mon-  
YYYY')  
FROM employees;  
# 15-Jun-2022
```

TO_DATE() – Convert String to Date

```
SELECT TO_DATE('15-06-2022', 'DD-  
MM-YYYY');  
# 2022-06-15
```

What is a JOIN?

A JOIN is used to combine rows from two tables based on a related column.

Here the common column is:

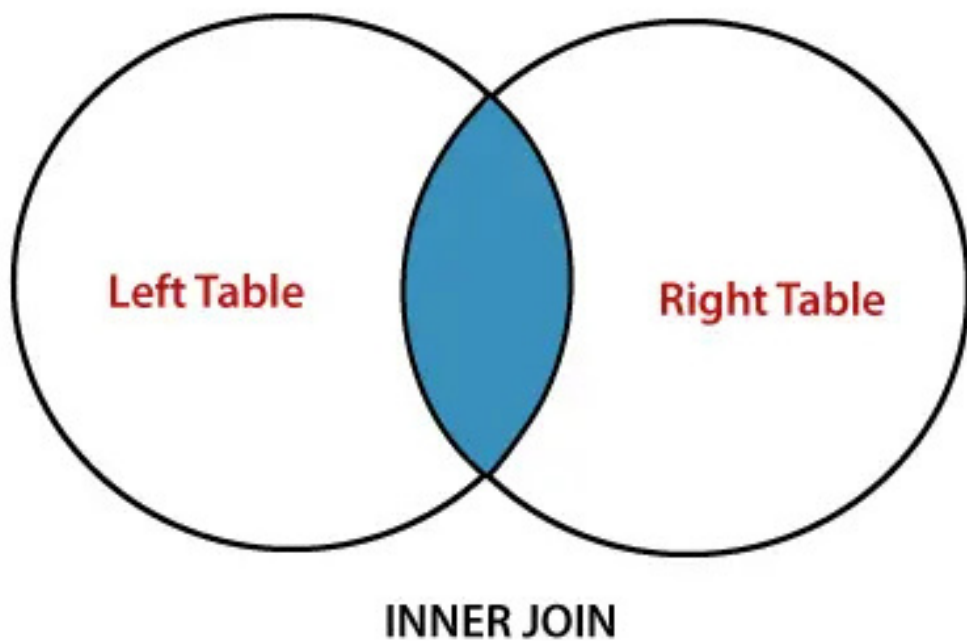
employees.dept_id = departments.dept_id

INNER JOIN (Most Important

Meaning

Returns only matching rows from both tables.

If there is no match → that row is excluded.



✓ Query

```
SELECT e.emp_name, e.salary,  
d.dept_name  
FROM employees e  
INNER JOIN departments d  
ON e.dept_id = d.dept_id;
```


Output

Rahul | 50000 | Sales

Neha | 60000 | IT

Amit | 55000 | Sales

Priya | 45000 | HR

Rohan | 70000 | IT

Notice:

Kiran is NOT included because dept_id is NULL.

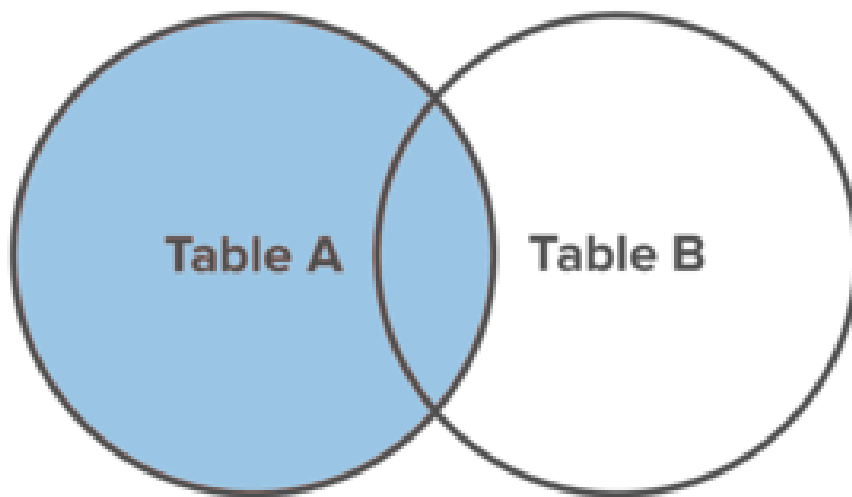
LEFT JOIN

Meaning

Returns:

- All rows from LEFT table (employees)
- Matching rows from RIGHT table (departments)

If no match → NULL



SQL LEFT JOIN

Query

```
SELECT e.emp_name, e.salary,  
d.dept_name  
FROM employees e  
LEFT JOIN departments d  
ON e.dept_id = d.dept_id;
```

Output

Rahul		50000		Sales
Neha		60000		IT
Amit		55000		Sales
Priya		45000		HR
Rohan		70000		IT
Kiran		40000		NULL

- ✓ All employees included
- ✓ Kiran has NULL department

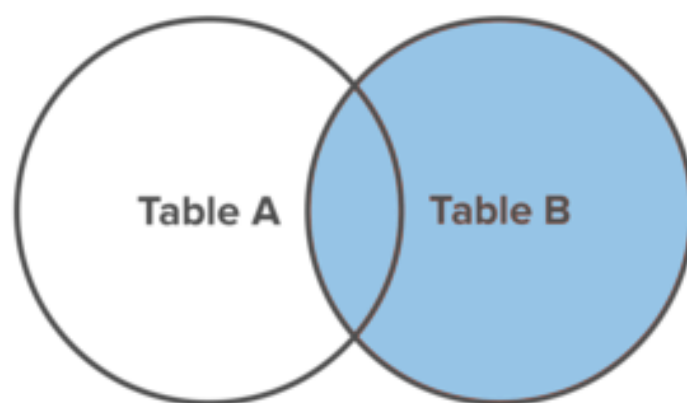
RIGHT JOIN

Meaning

Returns:

- All rows from RIGHT table (departments)
- Matching rows from LEFT table (employees)

If no employee in department → NULL



SQL RIGHT JOIN

Query

```
SELECT e.emp_name, e.salary,  
d.dept_name  
FROM employees e  
RIGHT JOIN departments d  
ON e.dept_id = d.dept_id;
```

Output

```
Rahul | 50000 | Sales  
Amit | 55000 | Sales  
Neha | 60000 | IT  
Rohan | 70000 | IT  
Priya | 45000 | HR
```

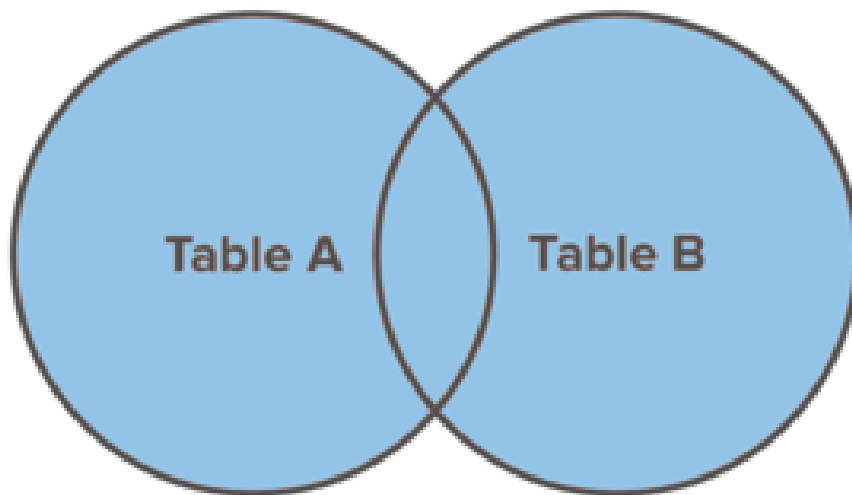
 Since all departments have employees, no NULL row appears.

FULL JOIN

Meaning

Returns:

- All rows from both tables
- If no match → NULL



SQL FULL JOIN

✓ Query

```
SELECT e.emp_name, e.salary,  
d.dept_name  
FROM employees e  
FULL JOIN departments d  
ON e.dept_id = d.dept_id;
```

✓ Output

```
Rahul | 50000 | Sales  
Amit | 55000 | Sales  
Neha | 60000 | IT  
Rohan | 70000 | IT  
Priya | 45000 | HR  
Kiran | 40000 | NULL
```

◆ Visual Understanding

INNER JOIN

Only matching data

LEFT JOIN

All employees + matching departments

RIGHT JOIN

All departments + matching employees

FULL JOIN

Everything from both sides

What is a Subquery?

A subquery is:

A query inside another query

Think like this:

“First find something → then use that result in the main query”

Real-life Analogy

Question:

“Show employees who earn more than the company **average salary**”

Step 1: Inner Query (Run this first)

```
SELECT AVG(salary) FROM  
employees;
```

Assume output:

55000

Step 2: Use that result in main query

```
SELECT emp_name, salary  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary) FROM  
employees  
);
```

✓ Output

Rahul | 60000

Rohan | 70000

👉 **This is a subquery.**

What is a Window Function?

A window function:

- Performs a calculation across related rows
- Does NOT reduce rows (unlike GROUP BY)
- Shows both row-level data and aggregated data together

👉 **This is the key difference.**

ROW_NUMBER() – Most Used

What it does

Gives a unique number to each row.

```
SELECT emp_name, salary,  
       ROW_NUMBER() OVER (ORDER BY  
salary DESC) AS row_num  
FROM employees;
```

Output

Rohan	70000	1
Neha	60000	2
Amit	55000	3
Rahul	50000	4
Priya	45000	5
Kiran	40000	6

RANK()

Use case:

Give unique number to each row

```
SELECT emp_name, salary,  
       RANK() OVER (ORDER BY salary  
DESC) AS rank_num  
FROM employees;
```

If two salaries are same:

Rank: 1, 1, 3

Output (if tie exists)

Neha | 60000 | 2

Amit | 60000 | 2

Rahul | 50000 | 4

👉 Skips rank numbers

DENSE_RANK() – No Gaps

```
SELECT emp_name, salary,  
       DENSE_RANK() OVER (ORDER BY  
salary DESC) AS dense_rank  
FROM employees;
```

If two salaries are same:

Rank: 1, 1, 2

👉 No gap

PARTITION BY (Very Important)

Question:

Rank employees inside each department

```
SELECT fname, department, salary,  
       RANK() OVER (PARTITION BY  
department ORDER BY salary DESC) AS  
dept_rank  
FROM em_full;
```

Output

Rahul	1	50000	2
Amit	1	55000	1
Neha	2	60000	2
Rohan	2	70000	1

Running Total (SUM with OVER)

```
SELECT emp_name, salary,  
       SUM(salary) OVER (ORDER BY salary)  
AS running_total  
FROM employees;
```

Output

Kiran	40000	40000
Priya	45000	85000
Rahul	50000	135000
Amit	55000	190000
Neha	60000	250000
Rohan	70000	320000



ADVANCED QUERY

```
SELECT fname, department, salary,  
       sum(salary) over (PARTITION BY  
department order by salary desc) as  
Running_profit  
FROM em_full;
```



OUTPUT

#	fname	department	salary	running_profit
	Rahul	Sales	70000	70000
	Amit	Sales	60000	130000
	Neha	Sales	50000	180000
	Rohan	IT	80000	80000
	Pooja	IT	70000	150000
	Kiran	IT	60000	210000

“SQL is not about memorizing queries, it is about understanding data and logic.”

**Grateful for the learning journey so far.
More practice, more projects, and
continuous improvement ahead 🚀📊**

**Thank you for reading all the chapters.
Learning, practicing, and growing every day 🚀📊**